

Chaîne d'approvisionnement logicielle · Python

PyPI, fichiers .pth et slopsquatting : le pip install n'est plus anodin

Le premier semestre 2026 a vu plusieurs paquets très largement utilisés être compromis à la source. Un cas Python mérite l'attention : une charge utile qui s'exécute au démarrage de l'interpréteur, sans que le paquet soit jamais importé.

Date de veille 25 juillet 2026

Rédacteur Sami El Abdallaoui, stagiaire veille et contenus

Déclencheur Nos exercices font installer des paquets aux apprenants

Public visé Équipe technique et pédagogique OpenCyberAI

Temps de lecture 7 minutes

À retenir

- Mars 2026 : plusieurs compromissions coordonnées de paquets légitimes — Trivy, KICS, LiteLLM et Telnix côté PyPI, Axios côté npm. Il ne s'agit pas de faux paquets, mais des vrais, empoisonnés à la publication.
- Sur LiteLLM, une des versions piégées passait par un fichier **.pth**, exécuté automatiquement au démarrage de l'interpréteur Python : aucun import nécessaire, et la vérification d'empreinte de pip reste valide.
- Les vagues de type ver observées ensuite fonctionnent en cascade : un jeton de mainteneur volé sert à republier des versions empoisonnées de *tous* ses paquets.
- Nouveau vecteur, le **slopsquatting** : les assistants de code inventent des noms de paquets qui n'existent pas ; il suffit à un attaquant d'enregistrer ce nom sur PyPI.
- Les parades sont connues et peu coûteuses : épingler les versions, verrouiller les empreintes, isoler l'environnement, ne jamais installer un paquet suggéré sans vérifier qu'il existe et qui le maintient.

Le cas LiteLLM, et pourquoi il est instructif

LiteLLM est une bibliothèque d'interface avec des modèles d'IA, très largement téléchargée. Fin mars 2026, deux versions publiées sur PyPI contenaient du code malveillant attribué à un groupe actif sur plusieurs écosystèmes. La première version embarquait sa charge dans un module exécuté à l'import — classique. La seconde est plus intéressante.

Le mécanisme du fichier .pth

Python lit au démarrage les fichiers d'extension **.pth** présents dans les répertoires de site-packages. À l'origine, ils servent seulement à ajouter des chemins au `sys.path`. Mais une ligne commençant par `import y` est exécutée. Trois conséquences :

- La charge se déclenche à **chaque** lancement de l'interpréteur sur la machine, même si le paquet n'est jamais utilisé.
- Le fichier est correctement déclaré dans le RECORD du wheel : le contrôle d'intégrité de pip passe sans rien signaler.

- La portée déborde le projet : tout l'environnement Python de l'hôte est concerné, ce qui maximise la collecte d'identifiants.

L'enseignement est net : la vérification d'empreinte garantit que l'on a bien téléchargé ce que le mainteneur a publié. Elle ne garantit rien sur ce que le mainteneur a publié.

Le slopsquatting, angle mort des assistants de code

Le typosquatting exploite les fautes de frappe. Le slopsquatting exploite les hallucinations : interrogés sur un problème, les assistants de code proposent régulièrement d'installer des paquets dont le nom sonne juste mais qui n'existent pas sur PyPI. L'attaquant n'a qu'à enregistrer ce nom et attendre. Il n'interagit jamais avec la victime.

Ce vecteur concerne directement notre public : un lycéen ou un étudiant qui suit une suggestion d'assistant et lance un `pip install` sans lever les yeux est exactement la cible.

Hygiène des dépendances : ce qui est applicable tout de suite

- Épingler les versions exactes dans `requirements.txt`, jamais de plage ouverte.
- Verrouiller les empreintes et installer avec `pip install --require-hashes`.
- Travailler systématiquement en environnement virtuel, et ne pas exécuter de code de projet avec l'interpréteur système.
- Avant toute installation d'un paquet inconnu : vérifier l'existence du dépôt, la date de la dernière publication, le nombre de mainteneurs et la correspondance entre la version PyPI et une release publiée sur le dépôt.
- En intégration continue, traiter les scanners de sécurité comme n'importe quelle autre dépendance : les épingler et les priver des secrets de production.
- Ne jamais installer un paquet uniquement parce qu'un assistant l'a nommé.

Ce que cela change pour nos contenus

— Audit à faire sur nos énoncés : plusieurs exercices demandent un « `pip install` » sans aucune précaution. Nous enseignons donc, en creux, un mauvais réflexe à des débutants.

— Proposition d'un module court « hygiène des dépendances » : trois étapes — repérer un paquet suspect, épingler une version, comprendre pourquoi le hash ne suffit pas.

— Le fichier `.pth` est un excellent support de scénario : il est concret, surprenant, et il fait comprendre la différence entre intégrité et confiance en dix minutes.

— Le slopsquatting est un sujet d'actualité immédiat pour un public qui utilise des assistants de code au quotidien.

Sources consultées

- Zscaler ThreatLabz — « Supply chain attacks surge in March 2026 »
- RapidFort — « PyPI, npm, and the New Frontline of Software Supply Chain Attacks », avril 2026
- Phoenix Security — analyse des campagnes npm / PyPI, juin 2026
- Stéphane Robert — « Sécuriser les dépendances npm, PyPI et Go », juillet 2026
- Documentation Python — site module, fonctionnement des fichiers `.pth`

Méthode : les faits techniques (versions concernées, mécanisme du .pth) ont été repris de rapports d'éditeurs de sécurité puis recoupés avec la documentation officielle de Python pour le fonctionnement des fichiers .pth. Les volumétries annoncées par les différents rapports divergent sensiblement et ne sont pas reprises ici.